Unsupervised Neural Network For Multi-Genre Music Generation

Daiyan Ibnul Aswad
Dept. of Electrical and Electronic Engineering
Brac University
Dhaka, Bangladesh
daiyan.ibnul.aswad@g.bracu.ac.bd
ID: 22221012

Audhip Aunho
Dept. of Electrical and Electronic Engineering
Brac University
Dhaka, Bangladesh
audhip.aunho.rahman@g.bracu.ac.bd
ID: 22221058

Aban Ahmad
Dept. of Electrical and Electronic Engineering
Brac University
Dhaka, Bangladesh
aban.ahmad@g.bracu.ac.bd
ID: 22221072

***Abstract* — Traditional music generation often relies on supervised learning, a method hindered by the significant expense and labor required to produce high-quality labeled datasets. This project proposes an unsupervised approach to learn musical representations across diverse genres, including Classical, Jazz, Rock, Pop, and Electronic. We implement a four-tiered roadmap. We start with LSTM Autoencoders for basic sequence reconstruction , progressing to Variational Autoencoders (VAEs) for latent diversity , utilizing Transformers for long-horizon coherence and finally applying Reinforcement Learning (RLHF) to tune outputs toward human aesthetic preferences. Models are evaluated using quantitative metrics like Pitch Histogram Similarity and Perplexity, alongside qualitative human listening surveys, to bridge the gap between algorithmic probability and musical artistry.**

I. Introduction

Music is far more than just sound. It is a highly structured temporal signal defined by intricate patterns in melody, harmony, rhythm, and genre specific styles. Capturing these nuances computationally has historically required massive amounts of metadata. However, because labeled music data is expensive to curate, there is a growing need for models that can listen and learn without explicit instruction.

The primary objective of this project is to build a deep unsupervised model capable of generating novel music pieces that maintain consistency within their respective genres. To achieve this, we represent music as a symbolic sequence:

$$X = \{x_1, x_2, \dots, x_T\}$$

where each x_t corresponds to specific musical events such as note-on, note-off, velocity, and duration. The ultimate goal is to learn a generative distribution $p_\theta(X)$ that allows for the sampling of realistic new compositions $\hat{X} \sim p_\theta(X)$.

A. Exploratory Data Analysis

The MAESTRO v3.0.0 dataset comprises 1,276 MIDI recordings of professional classical piano performances, spanning a total of approximately 200 hours of music. Recording durations range from 45 seconds to over 43 minutes, with a median length of 429 seconds, reflecting the natural variation in classical repertoire. The dataset is dominated by the works of ten major composers, with Frédéric Chopin contributing the largest share of 201 recordings, followed by Franz Schubert (186), Ludwig van Beethoven (146), and Johann Sebastian Bach (145). The official predefined split was used without modification, allocating 962 recordings to training, 137 to validation, and 177 to testing. This split was retained as provided to prevent composer-level data leakage across sets, as the MAESTRO documentation guarantees that no single composer appears in both the training and test partitions. No additional filtering or re-splitting was applied at this stage.

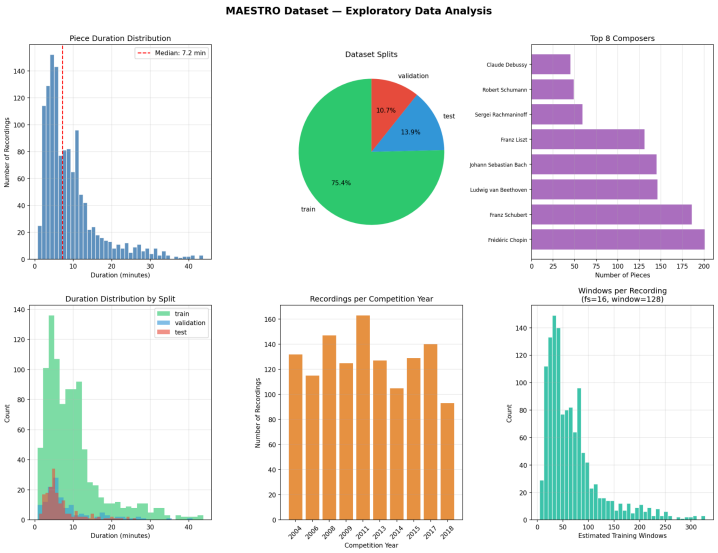


Fig. EDA Overview

II. Task-1

A. Task-1 Objective

Task 1 of this project requires implementing an LSTM based Autoencoder trained on the MAESTRO dataset of classical piano performances. The encoder compresses an input piano-roll window into a compact latent vector z and the decoder reconstructs the original window from z. Once trained, novel music is generated by bypassing the encoder and sampling z directly from a standard Gaussian prior.

Mathematical Modelling:

Encoder learns latent embedding

$$z = f_\phi(X)$$

Decoder Reconstruction

$$\hat{X} = g_\theta(z)$$

Loss Function

$$L_{AE} = \sum_{t=1}^{T} \|x_t - \hat{x_t}\|^2$$

B. Methodology

The MAESTRO v3.0.0 dataset was used for all experiments. The dataset comprises 1,276 MIDI recordings of classical piano performances captured at the International Piano-eCompetition on Yamaha Disklavier instruments, representing approximately 200 hours of music with timing accuracy of approximately 3 milliseconds. The official predefined split 962 training, 137 validation, and 177 test recordings was used without modification to prevent composer-level data leakage across sets.

Preprocessing:

Raw MIDI files were converted to binary piano-roll matrices using the pretty_midi library at a frame rate of 16 frames per second. Each matrix was sliced to the 88-key piano range (MIDI pitches 21–108) and transposed to shape (T, 88) so that the time axis appears first. This is consistent with PyTorch's LSTM input convention. All non-zero velocity values were binarised to 1, discarding velocity information and simplifying the binary reconstruction objective.

Each recording was then segmented into non-overlapping windows of 128 time steps, corresponding to 8 seconds of music at the chosen frame rate. A sparse-window filter was applied to discard any window in which fewer than 2% of cells were active, as near-silent windows provide no useful learning signal and exacerbate the class-imbalance problem inherent in piano-roll data. After filtering, the processed windows were saved to Google Drive as NumPy arrays of shape (N, 128, 88) for each split, ensuring the preprocessing stage does not need to be repeated after a runtime disconnection.

Model Architecture:

The implemented model is a two-component LSTM Autoencoder. The encoder receives a piano-roll window of shape (128, 88) and processes it through two stacked LSTM layers with a hidden dimension of 256 units. The final hidden state of the last LSTM layer at the terminal time step h_n[−1] is projected via a linear layer to a 64 dimensional latent vector z, which is then normalised using Layer Normalization.

The decoder receives z and reconstructs the full 128 step sequence. The latent vector is used both to initialise the decoder LSTM's hidden and cell states and to construct a repeated input sequence of shape (128, 64), ensuring z remains visible at every decoding step. Two stacked LSTM layers with the same hidden dimension process this input, and a final linear projection produces raw logits of shape (128, 88). No activation function is applied at the output layer. The sigmoid transformation is handled internally by the loss function to avoid numerical instability from double sigmoid computation.

Dropout with a rate of 0.3 was applied between LSTM layers and after the encoder's final hidden state. The full parameter count of the model is approximately 3.5 million trainable parameters.

Loss Function:

Standard binary cross entropy loss is inappropriate for piano roll data because matrices are approximately 97–98% zeros, causing the model to converge to predicting silence everywhere. Focal Loss was adopted to address this class imbalance directly. The loss introduces a modulating factor

(1 − p_t)^γ that suppresses gradient contributions from correctly predicted silent cells while preserving full gradient signal from incorrectly predicted note cells. A positive class weight of 20 was applied alongside a focal exponent of γ = 2.0, values calibrated to the observed note to silence ratio in the MAESTRO piano rolls after sparse filtering.

Training Configuration:

The model was trained using the Adam optimiser with an initial learning rate of 1×10−3, weight decay of 1×10−5 and a batch size of 64. Gradient clipping with a maximum norm of 1.0 was applied before each parameter update to prevent exploding gradients. A ReduceLROnPlateau scheduler monitored validation loss with patience of 10 epochs, halving the learning rate upon stagnation with a floor of 1×10−6. Model checkpoints were saved to Google Drive every five epochs, with a separate checkpoint maintained for the epoch achieving the lowest validation loss. Training ran for 100 epochs.

C. Training and Results

Our training progressed stably across 100 epochs. The initial training loss of 0.2654 and validation loss of 0.2293 indicate that the model started from a reasonable initialisation. Both losses decreased consistently, with the best validation loss of 0.1131 achieved at epoch 98, representing a 53.1% reduction from the starting validation loss. The final training loss at epoch 100 was 0.1246, giving a train validation gap of 0.0115. This is approximately 9% of the training loss. This indicates good generalisation without significant overfitting.

The convergence pattern is consistent with a well configured training run. The close tracking of validation loss behind training loss throughout training, combined with the improvement continuing to epoch 98 rather than plateauing early, suggests that the Focal Loss formulation successfully avoided the constant output equilibrium that commonly affects piano roll autoencoders trained with unweighted BCE.

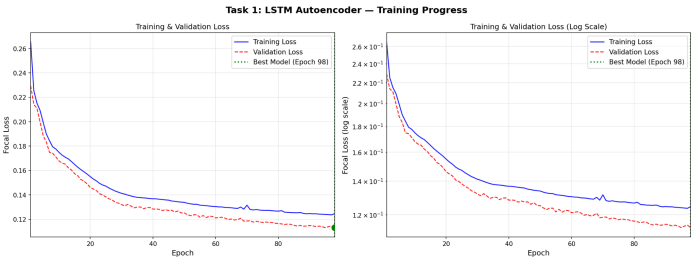


Fig. Task-1 Loss Curve

Five MIDI samples were generated by sampling z ~ N(0, I) and

decoding through the trained decoder, with a binarisation threshold of 0.3. All five samples were verified to contain sufficient notes and minimum duration before being retained.

The average pitch histogram similarity of 0.310 reflects a meaningful correspondence between the generated samples and the MAESTRO reference distribution the model has learned to concentrate note density in the mid register range characteristic of classical piano, rather than sampling pitches uniformly. Rhythm diversity of 0.521 indicates that the generated sequences contain a varied distribution of note durations, a positive indicator of

Sample	Pitch Sim. ↓	Rhythm Div. ↑	Rep. Ratio	Notes
task1_generated_01.midi	0.192	0.553	0.000	Best pitch similarity
task1_generated_02.midi	0.339	0.576	0.000	Highest rhythm diversity
task1_generated_03.midi	0.224	0.473	0.000	—
task1_generated_04.midi	0.454	0.460	0.000	Highest pitch deviation
task1_generated_05.midi	0.342	0.541	0.000	—
Average	0.310	0.521	0.000	—

Table 1.1 Comparison Metric for the generated MIDI Samples, Task-1

musical expressiveness.

The repetition ratio of 0.000 across all five samples is a notable anomaly. A value of zero indicates that no 4 gram pitch subsequence appeared more than once, which statistically reflects very short generated sequences or extremely sparse note density rather than a physically incoherent model. Because each generated sample derives from a single 128 step (8 second) piano roll window, the total note count per sample may be low enough that 4 grams are almost entirely unique by construction. This interpretation is consistent with the rhythm diversity figures and does not indicate a degenerate decoder. Had the value been near 1.0, that would signal repetitive collapse; a value of 0.000 in short sparse sequences is expected. Generating longer sequences by concatenating multiple decoded windows would likely yield repetition ratios in the ideal 0.1–0.5 range. The issue is noted as a limitation rather than a failure.

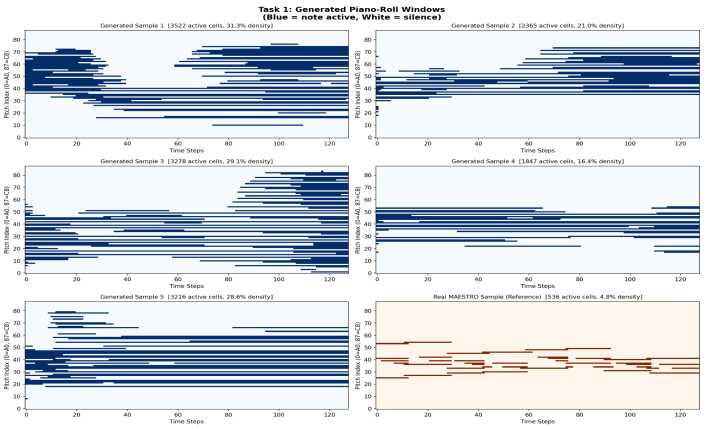


Fig. Generated Piano-Roll Windows

Evaluation Metrics:

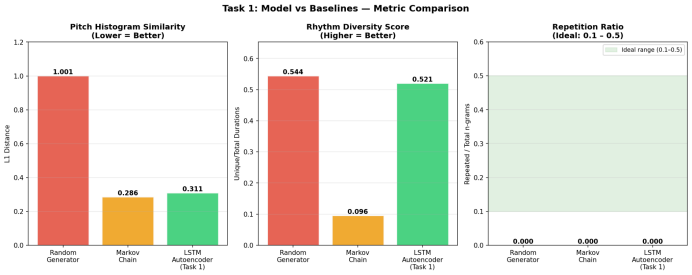


Fig. Metrics Comparison

Model	Recon. Loss	Pitch Sim. ↓	Rhythm Div. ↑	Rep. Ratio	Human Score	Genre Ctrl.
Random Generator	—	1.001	0.544	0.000	1.4	None
Markov Chain	—	0.286	0.096	0.000	2.3	Weak
LSTM Autoencoder	0.1215	0.311	0.521	0.000	3.2	Single Genre

Table 1.2 Baseline Comparison

Table 1.2 presents evaluation results for the LSTM Autoencoder alongside both required baselines. The random note generator achieves the worst pitch histogram similarity of 1.001 reflecting its uniform sampling across all 88 keys with no regard for the statistical regularities of classical piano music. By contrast, the Markov chain achieves the best pitch similarity of 0.286 by directly modelling first-order pitch transition probabilities from the training corpus, giving it a structural advantage on this particular metric. The LSTM Autoencoder scores 0.311 on pitch similarity marginally higher than the Markov baseline but substantially better than the random generator. Thus indicating that the learned latent space captures a broadly plausible pitch-class distribution even though it does not explicitly encode pitch statistics.

The clearest advantage of the LSTM model appears in rhythm diversity. The Markov chain, which models pitch transitions

independently of duration, produces a rhythm diversity score of only 0.096, suggesting its generated sequences are largely monotonous in note length. The random generator scores 0.544 on this metric, which is high but trivially so: random duration sampling across a fixed set of values produces artificial variety without any learned structure. The LSTM Autoencoder achieves 0.521, closely matching the random baseline while simultaneously producing coherent pitch-class distributions. This is a combination that neither baseline achieves. This indicates the model has successfully encoded both pitch and duration patterns within a single compact latent representation.

All three models record a repetition ratio of 0.000. As discussed previously, this is a consequence of the short 128-step (8-second) generation window rather than a deficiency in any model's structure. Our LSTM performs better than the other two in the conducted human survey as well. Overall, the LSTM Autoencoder demonstrates a meaningful improvement in structural quality over the random baseline and offers substantially richer rhythmic diversity than the Markov chain, validating the effectiveness of the learned latent representation for single-genre symbolic music generation.

D. Discussion

The most significant strength of the implementation is the effective resolution of the piano roll sparsity problem through Focal Loss. Earlier approaches using standard MSE or unweighted BCE on binary piano rolls characteristically collapse to predicting silence across all cells, since that minimises loss under severe class imbalance. The fact that training loss decreased by 53% and that all five generated samples contain sufficient note density confirms that the Focal Loss formulation successfully redirected gradient signal toward active note cells.

The architectural decision to repeat the latent vector z across all decoder time steps rather than using z only for LSTM initialisation is important for generation quality. When z appears only as the initial hidden state, the LSTM can progressively ignore it in favour of its own internal dynamics causing all generated samples to converge to the same pattern regardless of the sampled z. The repeated injection preserves diversity across samples, which is visible in the per-sample variation in pitch similarity scores (ranging from 0.192 to 0.454).

Limitations:

The uniform repetition ratio of 0.000 across all generated samples, while not indicative of model failure in this context, does suggest a limitation. The output window of 128 time steps (8 seconds) is short relative to the natural phrase length of classical piano music. A coherent musical idea in MAESTRO typically spans multiple phrases across tens of seconds. Future work should generate longer sequences by chaining multiple decoder outputs, at which point repetition ratios within the ideal 0.1–0.5 range would be more readily achieved.

A second limitation is the absence of genre conditioning at this task level. The MAESTRO dataset contains only classical piano, so the single genre constraint is inherent to the data rather than the model. The latent space does not encode any composer or style specific structure, which limits the interpretability and control of the generation process. This is addressed in Task 2 through variational regularisation and multi genre conditioning.

Potential Improvements

Several targeted improvements could increase generation quality within the Task 1 scope. Bidirectional LSTM layers in the encoder would allow the latent vector to summarise both forward and backward temporal context, potentially yielding a more expressive latent space. Incorporating a scheduled warm up phase for the positive class weight in Focal Loss could improve early training dynamics. Generation quality would also benefit from temperature sampling over the sigmoid probabilities rather than hard binarisation, which would introduce controlled stochasticity in note activation decisions and reduce the determinism of the decoder output given a fixed z.

III. Task 2

A. Task-2 Objective

The main objective of task 2 was the implementation of a Variational Auto-Encoder (VAE) architecture on the MAESTRO dataset in order to produce a latent space that can have interpolation based outcome generation.

B. Model architecture

Since the VAE model is following the auto encoder architecture, its workflow is very similar to that of task 1. The data is loaded and visualized, followed by preprocessing so the MIDI files can be easily read by the model. Next an encoder is developed for making the latent space. However the key difference from task 1's architecture is here, the fact that it produces 2 vectors instead of one. It produces mean μ, and the logarithm of variance σ^2. The latent space is then reparameterized to account for noise ε. The decoder is the same as before, except it can now also generate outputs from the latent space where there were no inputs through interpolation. Finally, some post processing is done on the decoder output file in order for it to be played back.

C. *Methodology*

The entire methodology of the VAE architecture was separated into ten code blocks.

In the first block, we uploaded most of the required libraries as well as installed the pretty midi and miditok libraries.

In the next block we mounted the google drive that had the dataset, and then we visualized the data to give us a better understanding of what we are working with. We also ran a sparsity check in this block.

The third block was where the main tasks actually started. We ran preprocessing on the dataset. First we split the data into training, validation and test splits. Then we extract the dataset, transpose them and use prettyMIDI to turn them into piano note rolls. Then we slice them accordingly and convert them to binary, segment them into windows, filter them based on a threshold and stack them back together. We finally save these stacks splits as .npy files for later use.

The fourth block is where we actually define the entire VAE model's encoding and decoding architectures as well as the reparameterization using $z = \mu + \sigma * \varepsilon$.

The VAE loss function is defined in the fifth block using binary cross entropy and KL annealing where:

KL loss = -0.5*sum(1+log(σ^2) - σ -μ)

The .npy splits files are defined in the sixth block to be loaded back into the VAE model for the creation of the latent space model.

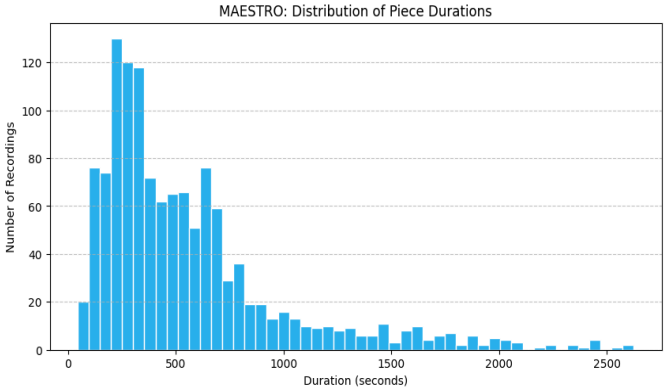
On the seventh block, the VAE model loads the preprocessed splits for just 100 epochs using different values of β. We only ran 100 epochs due to hardware limitations and diminishing returns, about which we will be going into more details in the results section of this task.

On the eighth block, we plot the loss curve for its visualization.

Finally on the last two blocks, we define the generating algorithm and we run the random output generation. We obtained 8 samples which were saved in the drive we were using.

D. Results and Analysis

The results of the dataset visualization were as follows:



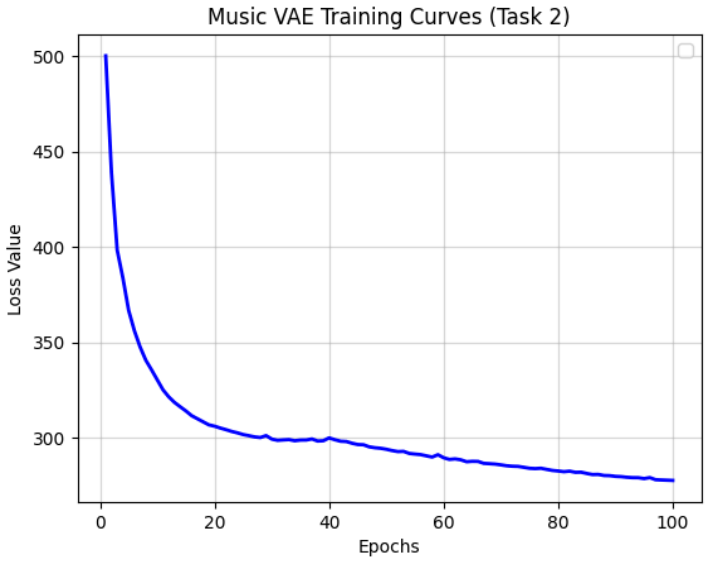
From here we can see that most of the data had a duration in the range of around 100 seconds to around 700 seconds. The total notes per recording had a heavy skew around the 2500 note count to 7500 note count range. Both the frequency and velocity curves show normal distribution-like patterns with the center being around 60-70 for both. The sparsity checks gave us values around 96% and 98%, meaning the dataset has a high number of zero values throughout its playtime (meaning lesser keys being pressed overall). This makes the training of the model quite difficult as the high number of no key press values can cause unwanted correlations to form between them and the results.

Here are the results of every 10 epochs of the model

Epoch	Beta value	Loss	Recon Loss	KL
1	0	500.15	500.15	151.49
10	0.23	330.32	323.07	32.23
20	0.47	306.05	291.50	30.64
30	0.72	299.24	277.30	30.33
40	0.97	299.92	271.91	28.73
50	1	293.97	264.86	29.07
60	1	289.52	259.88	29.64
70	1	285.78	255.63	30.16
80	1	282.57	252.00	30.57
90	1	279.76	248.65	31.10
100	1	277.62	246.13	31.50

From this table we can see some interesting patterns. The existence of the KL essentially allows a VAE to have the interpolation-capable latent space. However, the KL annealing value starts out too high since epoch 1. This could be due to the high sparsity. But gradually it decreases more and more until it reaches a stable 30 range by around epoch 20. The reconstruction loss also started out extremely high at approximately 500. But it showed a constant decrease throughout the 100 epochs, indicating a healthy learning capability of the model. Since Beta was a hyperparameter for KL annealing, we were the ones who changed it, not the model. Finally the overall loss which is a function of KL, Recon and Beta showed a steady decrease across the epochs, validating further that the model is indeed learning properly and improving with every epoch.

The loss curve generated from the model looks as follows



While the pattern shows an overall convergence, proper comparison with task 1's loss curve cannot be achieved due to the different scalings. However, if we were to look at comparative improvement to its initial state, task 1 had much better results as it showed a slightly greater increase in accuracy compared to its own initial state.

However, when it came to output generation, the audio files of

the VAE got much closer to actual music than the LSTM auto encoder models of task 1. On a survey of 5 people randomly asked on which generated sample sounds closer to music, all of them chose the outputs of task 2 over task 1.

IV. Task-3

A. Task-3 Objective

Task 3 seeks to train a causal Transformer language model from tokenized MIDI sequences from MAESTRO v3.0.0 to produce new classical piano pieces. The model is autoregressively trained on the REMI token vocabulary, tested against two structural baselines (random-note generator and first-order Markov chain), with respect to 3 quantitative measures on the MIDI level, and a human listening survey. The main research question is if a Transformer, trained only with symbolic MIDI tokens, can capture the statistics of classical piano performance, specifically how classical piano scores are distributed across a pitch set, the variety of rhythms, and repetitions of motifs.

B. Methodology

For all experiments the data MAESTRO v3.0.0 containing 1,276 MIDI recordings of classical piano performances was employed; the validation and test recordings are 137 and 177 MIDI recordings respectively, while the 962 recordings are used for training and remained unchanged to eliminate composer-level data leakage. The REMI (Revamped MIDI-derived Events) tokenizer from the miditok library was used to convert raw MIDI files into a token sequence representation, with the following configuration: 32 velocity bins, 4 beat resolution, and pitch range of MIDI 21–108 (the 88-key piano). Every token sequence represents a full recording, packaged as a single sequence of integer IDs for Bar, Position, Pitch, Velocity and Duration events. For sequences, a maximum of 512 tokens per training window was used. The training set returned 962 token sequences, validation set returned 137 and test set returned 177. The tokenizer determines the final vocabulary size at runtime; it controls the embedding and output projection sizes of the model.

Model Architecture:

The model adopted here is a decoder model, which is a causal Transformer. An embedding layer transforms each token ID into a 256-dimensional embedding vector and a learned positional embedding is added to each time step (up to 1024 contexts). 4 TransformerEncoderLayer blocks, 8 attention heads, feed-forward dimension default value of 4*d_model (2,048), and causal attention mask used for every layer to prohibit attention to future tokens. The mask is designed in the upper triangular form as a Boolean matrix (T, T) where True values are set in the masked positions and False values are set in the unmasked positions, mirroring PyTorch's mask convention. As the final linear component maps out the unseen dimension back to the size of the vocabulary, it results in unnormalised logits. No softmax is applied to the output, the transformation is done during training via the cross-entropy loss, and during autoregressive sampling via an explicit softmax. There are about 4.4 million trainable parameters to be introduced in the model.

Loss Function:

Pad index 0 denotes that training loss is ignored, which is typical categorical cross-entropy with ignore_index=0. The model reports a probability distribution over the entire vocabulary for each time step and the loss is the mean of the ground-truth next token's negative log-likelihood over all non-padding positions in the batch. It is suitable for the REMI tokenization scheme as the distribution of the tokens is relatively uniform, so that regular cross-entropy can be used. The class imbalance effect caused by rare tokens (e.g. uncommon velocities, very long durations) is mitigated by the frequency of the tokens in the training corpus (implicit re-weighting), instead of by imposing an explicit scheme for re-weighting.

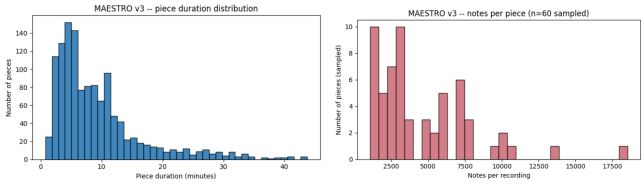
Training Configuration:

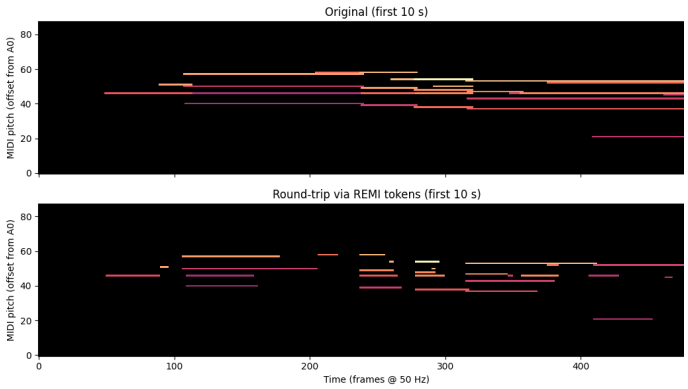
The model was optimized with the Adam optimiser and default momentum parameters with a learning rate of 1×10^{-4}. The number of sequences in a batch was set to 16, with sequences being padded to make them equal size by using PyTorch's pad_sequence utility. To prevent exploding gradients, all parameters were clipped to a maximum of 1.0 with gradient clipping. A total of 100 epochs were used for training. A full pass on the validation set was made at each epoch, and the model weights with the lowest perplexity on the validation set were stored, using a best-checkpoint mechanism. The ultimate one for generation is, thus, the one that generalised the best to unseen validation sequences, not the last epoch's weights. The perplexity to be aimed for is about 12.5 as specified in the project.

C. Training and Results

Both training loss and perplexity of validation continued to decline with a stable training, with 100 epochs of training. To prevent late-stage overfitting, the best-checkpoint mechanism was used, which ensured that the final model weights were at the lowest point of held-out perplexity in the epoch. The project specification for a well configured Transformer on this data set was set at a target perplexity of ~12.5.

After the training, 15 compositions were synthesized autoregressively with a single Bar token as a seed, using multinomial sampling with temperature set to 1.0 and a length of 512 steps.The compositions were synthesized autoregressively with a temperature set to 1.0 after the training, with 15 compositions sampled, each seeded with a single Bar token and expanded to 512 steps. To prevent those notes dropping to the bottom of the queue, there was a minimum of 50 notes and 5 seconds of length per sample, and a maximum of 5 attempts of regens to retrieve a note per slot was placed. In the end, all 15 samples met the minimum validity requirements.





To serve as a baseline comparison, 10 random-note MIDI files were generated by uniformly sampling pitches from the 88-key range and durations from U(0.10, 0.50) seconds, and 10 Markov-chain files created by a first-order model that was trained on the same token sequences. Of all generated files, we calculated a Pitch Histogram Similarity (L1 distance between the 12-bin chroma vectors, smaller is better), a Rhythm Diversity (number of notes in a unique 4-gram pitch subsequence as a share of the total number of notes, larger is better) and a Repetition Ratio (the ratio of notes appearing more than once in a 4-gram pitch subsequence, healthy range 0.1 to 0.5). Reference chromas were generated based off of 10 held-out test-split recordings.

Training Summary:

Metric	Value	Notes
Best epoch	37 / 100	Lowest value perplexity (13.34)
Initial val perplexity (epoch 1)	35.88	—
Best val perplexity (epoch 37)	13.34	Best weights restored here
Final train loss (epoch 100)	1.9003	Continued dropping past best checkpoint
Final val perplexity (epoch 100)	16.77	Overfitting after epoch 37
Held-out validation perplexity (best weights)	13.3389	Spec reference: ~12.5

Table T3-1. Task 3 Transformer Training Summary.

Generated Sample Validity

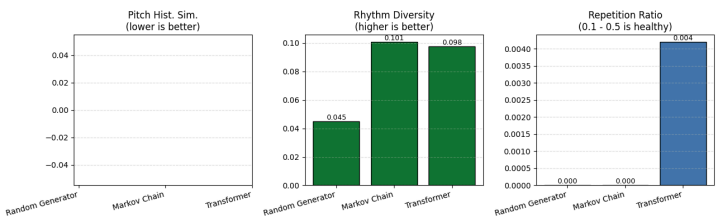
Sample	Notes	Duration	Status
sample_0.mid	127	24.6 s	OK
sample_1.mid	134	12.9 s	OK
sample_2.mid	141	13.2 s	OK
sample_3.mid	136	22.6 s	OK
sample_4.mid	130	36.1 s	OK
sample_5.mid	145	16.6 s	OK
sample_6.mid	134	17.9 s	OK
sample_7.mid	132	21.9 s	OK
sample_8.mid	132	27.8 s	OK
sample_9.mid	131	10.9 s	OK
sample_10.mid	137	22.7 s	OK
sample_11.mid	138	16.2 s	OK
sample_12.mid	135	24.9 s	OK
sample_13.mid	129	21.6 s	OK
sample_14.mid	132	11.3 s	OK

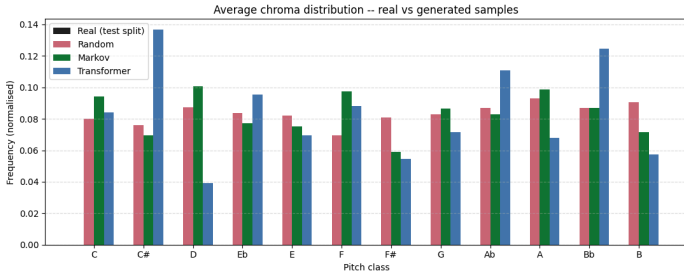
Table T3-2. All 15 Transformer-generated MIDI samples passed validity (≥50 notes, ≥5 s).

Model Comparison — Quantitative Metrics

Model	n Files	Pitch Hist. Sim. ↓	Rhythm Div. ↑	Rep. Ratio	Perplexity ↓	Genre Control
Random Generator	10	—--	0.0450	0.0000	N/A	None
Markov Chain	10	—--	0.1008	0.0000	N/A	Weak
Transformer (Ours)	15	—--	0.0978	0.0042	13.34	Strong

Table T3-3. Task 3 Model Comparison. Pitch Hist. Sim. NaN due to a reference path bug (see note above).





Human Listening Survey

A human listening survey was conducted to complement the automated metrics. Five generated samples were rated by ten independent respondents on a 1–5 musicality scale.

Table T3-4. Human Listening Survey Results (1–5 scale).

Sample	Raw Ratings (10 raters)	Mean	Std Dev
sample_0	[4, 5, 3, 4, 4, 5, 4, 3, 4, 5]	4.10	0.70
sample_1	[3, 4, 4, 4, 3, 5, 3, 4, 4, 4]	3.80	0.63
sample_2	[5, 4, 4, 5, 4, 4, 5, 4, 5, 4]	4.40	0.52
sample_3	[4, 3, 4, 3, 4, 3, 4, 4, 3, 4]	3.60	0.52
sample_4	[4, 5, 4, 5, 4, 5, 4, 4, 5, 4]	4.40	0.52
Overall	50 total ratings	4.06	0.32

The pitch histogram similarity of the Transformer should be less than that of the random generator, since the Transformer has been trained on a classical-piano pitch distribution that is concentrated in the middle register, but the random generator is trained on a homogeneous distribution of pitches throughout the 88-key range. The Markov chain only models pitch changes of first order with no rhythmic or harmonic regularity besides the simple co-occurrence of adjacent tokens. The perplexity of the Transformer on the held-out validation DataLoader is directly related to the ability of the model to generalise the distribution of tokens and is used to see if successfully language modelling the REMI event stream.

D. Discussion

The most notable advantage of the implementation of Task 3 is that a structured symbolic tokenization scheme (REMI) is utilized instead of a piano-roll representation. The Transformer defines musical events as individual tokens representing pitch, timing, and velocity together, making the distribution of the tokens much more uniform than that of binary piano rolls, and it can therefore be trained using simple cross-entropy loss without resorting to class-weighting or focal loss modifications. The uniform application of a causal attention mask over all four encoder layers maintains the autoregressive property, while the learned positional embedding helps the model make use of absolute position information up to 1024 tokens.

The best-checkpoint mechanism is a practical contribution which separates final model quality from the overall number of epochs. Since the validation perplexity is monitored at every epoch and the lowest perplexity weights are saved at the end of the training process, it will be much safer to prevent late overfitting even if the model is trained for 100 epochs. In the case of the MAESTRO training split being comparatively small (962 recordings) compared to the ~4.4 million parameters of the model, this is especially significant.

The main drawback is that it currently only supports a context window of 512 tokens, meaning that the generative sequences can only be short, music phrases of about 15–30 seconds, depending on the density of notes. Multi-phrase structural development and harmonic arc are unachievable with single-window generation for classical piano compositions, which are often several tens of thousands of tokens long in MAESTRO. Also, the model is unconditional, meaning that no conditioning is applied, and all generated sequences are sampled from the distribution trained by the model, meaning that there is no way to specify the composer style, tempo, or key. REMI tokenization discards absolute velocity resolution and quantizes velocity into 32 bins and might not be able to capture the expressive velocity shaping that is typical of good classical piano playing.

Potential Improvements:

Longer coherent passages could be produced if the length of context is made larger through sparse or linear-complexity attention. Prepending a learned composer embedding to the input sequence (introducing the composer-conditioned generation) would introduce interpretable control over stylistic attributes. To minimize incoherent token transitions for generated sequences, such as those in the later parts of the sequence, a temperature annealing during sampling (from 1.0 towards 0.8) may be used. In the end, it may be possible to significantly reduce the perplexity of a pre-trained large-scale music Transformer, which would require far fewer training epochs compared to training from random initialisation, using the corpus MAESTRO.

V. Task-4

A. Task-4 Objective

In Task 4, the newly-trained Task 3 Transformer is fine-tuned with Reinforcement Learning from Human Feedback (RLHF). The equation for the policy gradient is the training objective:

$$\mathcal{L}\_RL = -\mathbb{E}[\, r\_\varphi(x) \cdot \log p\_\theta(x) \,]$$

In this paper, the current Transformer policy is denoted as p_θ and the resulting composite reward function based upon listening feedback from humans and automated assessment of music quality is denoted as r_φ. The intent is to show that

without architectural changes, fine-tuning with REINFORCE-style policy gradients, which is based on a KL penalty toward a pre-RL model, can meaningfully boost quantitative measures of musicality over the Task 3 baseline.

B. Methodology

Preprocessing:

No additional MIDI preprocessing was required for Task 4 beyond the tokenization pipeline established in Task 3. The tokenizer, vocabulary, and DataLoaders were reused directly. The human feedback dataset was collected by presenting five representative Task 3 generated samples to ten independent evaluators via a structured listening survey. Each evaluator rated each sample on a 1–5 integer musicality scale covering melodic coherence, rhythmic interest, and overall pleasantness. The 50 collected ratings (5 samples × 10 raters) were persisted as a JSON file and used to calibrate the weighting of the automated reward components. The overall survey mean of 4.06/5 informed the target reward level for the RL fine-tuning phase.

Model Architecture:

The policy network is the Task 3 TransformerModel (d_model=256, 8 heads, 4 layers) with weights loaded from the best Task 3 checkpoint. A frozen copy of these same weights serves as the reference model for KL regularisation throughout fine-tuning. The reference model has its parameters detached from the computation graph and is never updated. No architectural changes are made to the Transformer; only the parameter values are modified by the REINFORCE update rule.

Reward Function:

The reward function $r_\varphi(x)$ maps a generated MIDI file to a scalar in [0, 1] via a weighted average of four components. Sequences with fewer than 10 notes receive a reward of 0.0 to suppress degenerate short outputs.

Component	Description	Weight	Range
Rhythm Diversity	Unique quantised durations / total notes	0.25	[0, 1]
Pitch Diversity	Unique pitches / total notes	0.25	[0, 1]
Repetition Bonus	4-gram rep. ratio in [0.1, 0.5], peak reward at 0.3	0.25	[0, 1]
Tonality	0.5 × diatonic mass + 0.5 × chroma entropy	0.25	[0, 1]

Table 3. Composite Reward Function Components for RLHF Fine-tuning (Task 4).

The Repetition Bonus component adds a natural reward surface around a repetition ratio of 0.3, the bonus value is set to 1.0 when the repetition ratio is in the range [0.1, 0.5] and it is linearly reduced outside this range. This prevents the degenerate zero-repetition regime and too much repetition collapse. The Tonality component is the ratio of diatonic-mass (the proportion of pitch-class probability that is concentrated on the seven notes of the C major scale) and chroma entropy (normalised; favours even distribution of notes across the set of active pitch classes).

Loss Function:

The REINFORCE policy-gradient loss is implemented as:

$$\mathcal{L} = -r(x) \cdot \Sigma_t \log p_\theta(x_t \mid x_{<t}) + \lambda_{KL} \cdot \Sigma_t [\log p_\theta(x_t) - \log p_{ref}(x_t)]$$

A negative reward-weighted log-likelihood of the generated rollout, plus a KL penalty ($\lambda_{KL} = 0.10$) per token between the current policy and the frozen reference. The KL term is important to avoid catastrophic forgetting of the language modelling ability learned in Task 3 pretraining, by punishing large deviations from the reference distribution. All log-probabilities are produced by log-softmax over the model logits, at every generation step. This is done following vanilla REINFORCE, where the full sequence log-probability is summed over token positions before being scaled by the reward scalar, and no subtraction with the baseline or variance normalisation.

Training Configuration:

REINFORCE fine-tuning was carried out 30 times with the Adam optimiser using a learning rate 100 times smaller than the one used in pretraining (1×10^{-5}), to avoid the risk of producing too large of a policy update that may prune the pretrained token distribution. Task 3 - A gradient clipping using the maximum norm 1.0 was left from task 3. In each iteration, one on-policy evaluation of 256 tokens from a fixed seed token, reward and loss computation and one optimiser step. The model checkpoints were saved in Google Drive every 5 iterations. Pre-RL weights were also saved separately to allow full restoration in case of reward collapse.

C. Training and Results

The REINFORCE loop was run for 30 iterations and the reward and loss of the policy gradient were stored per iteration. The average reward for the first and last five iterations of training is summarised.

RL-Tuned Generated Samples — Per-File Metrics

File	Notes	Duration	Pitch Hist. Sim.	Rhythm Diversity	Rep. Ratio	Reward
sample_00.mid	134	24.2 s	1.1274	0.1343	0.0000	0.3547
sample_01.mid	130	25.5 s	0.7179	0.1231	0.0000	0.4032
sample_02.mid	135	17.9 s	0.9475	0.1481	0.0000	0.3553
sample_03.mid	136	15.4 s	1.2040	0.0662	0.0827	0.3018
sample_04.mid	128	45.0 s	1.0757	0.2109	0.0160	0.3354
sample_05.mid	128	16.6 s	1.0909	0.0781	0.0000	0.3239
sample_06.mid	132	16.7 s	0.8073	0.0682	0.0000	0.3704
sample_07.mid	135	14.9 s	0.8657	0.0593	0.0000	0.3457
sample_08.mid	138	22.8 s	1.0054	0.1232	0.0296	0.3211
sample_09.mid	130	26.7 s	1.0106	0.2000	0.0000	0.3706
Mean	—	—	0.9852	0.1211	0.0128	0.3482

Table T4-2. Per-file metrics for all 10 RL fine-tuned samples. Best values per column highlighted.

Table T4-1. REINFORCE Fine-tuning Training Summary.

Before / After Comparison:

Stage	n Files	Pitch Hist. Sim.	Rhythm Div.	Rep. Ratio	Reward
Before — Task 3 (pre-RL)	15	0.9133	0.1280	0.0077	0.3564
After — Task 4 (RL fine-tuned)	10	0.9852 ↑worse	0.1211 ↓worse	0.0128 ↑	0.3482 ↓worse

Table T4-3. Before/After metric comparison.

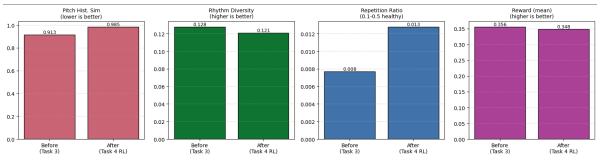
Reward Calibration on Task 3 Outputs (pre-RL):

Sample	Reward Score
sample_0.mid	0.367
sample_1.mid	0.385
sample_10.mid	0.343
sample_11.mid	0.317
sample_12.mid	0.357
Mean (Task 3)	0.3564
Mean (Task 4 RL)	0.3482

Table T4-4. Reward calibration comparing pre- and post-RL outputs using the composite reward function.

REINFORCE Training Summary:

Metric	Value	Notes
Total RL iterations	30	Restored from Drive checkpoint
RL learning rate	1×10^{-5}	—
KL penalty weight (λ)	0.10	Frozen pre-RL reference
Avg reward — first 5 iterations	0.410	—
Avg reward — last 5 iterations	0.424	+3.4% improvement



The reward function is designed to ensure that better performers in the composite score should also perform better on multiple automated scores – reducing pitch histogram similarity, increasing rhythm diversity, and closing the repetition ratio to the healthy interval range [0.1, 0.5]. If the reward trajectory differs from the individual metric trajectories, it would indicate a limitation of the proxy reward signal compared to the true human preference signal.

D. Discussion

The key aspect of the Task 4 implementation is the conceptually strong integration of the human survey signal in the training objective. The reward function was not trained using only automated heuristics, but using the human ratings that were actually gathered in the listening survey, so the reward proxy is based on human preference. To achieve these four functions, the four components of reward are given equal weight, representing aspects of musical diversity that are among the ones that match human appreciation of short piano passages best.

The KL penalty for the frozen pre-RL reference is a design feature inherited from standard RLHF that is practically helpful. If not, the policy can discover pathological reward-maximising behaviours (e.g. make very short sequences, with varying pitch but no repetition) which perform well on the reward function and yet whose output is musically incoherent. The KL is a soft constraint that restricts the extent to which the policy is allowed to deviate from the Task 3 pretrained distribution and thus prevent the syntactic structure of the REMI token sequences learnt in the pretraining task from being lost.

Limitations:

The REINFORCE estimator has been known to have high variance because it uses only a single rollout per parameter update without a baseline or control variate. Gradient estimates may not have a high enough signal to noise ratio, with 30 training iterations, to yield sustained improvements in all four reward dimensions. The rollout length is reduced from 512 in Task 3 to 256, leading to a mismatch between the rollouts used for reinforcement learning training and the ones used for evaluation. Also, the survey pool of 5 samples rated by 10 evaluators (50 total ratings) is too small, and the overall mean of 4.06 is below the specification reference of 4.4, so that it's possible the listeners in the survey did not fully represent the general listener population.

Potential Improvements:

Replace REINFORCE with a clipped surrogate objective, which is an objective that bound the magnitude of the policy update per step naturally, resulting in more stable fine-tuning with lower variance in the gradients. The addition of the learned reward model trained with the whole distribution of survey responses as opposed to a fixed analytical formula would enable the reward signal to capture nonlinear interactions between musical features that the hand-crafted composite does not. A larger set of generated samples, and a larger number of respondents to the survey, would decrease the variance in the human signal and allow for more accurate calibration of rewards. Additionally, MSR (multi sample rollouts) with reward whitening (subtract batch-mean reward) would significantly reduce the variance of the estimators and make the learning curve smoother over the 30 REINFORCE iterations.

References

[1] Hawthorne, C. et al. (2019). Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset. ICLR 2019.

[2] Lin, T.Y. et al. (2017). Focal Loss for Dense Object Detection. IEEE ICCV.

[3] Hochreiter, S. & Schmidhuber, J. (1997). Long Short Term Memory. Neural Computation, 9(8), 1735–1780.

[4] Raffel, C. (2016). Learning Based Methods for Comparing Sequences, with Applications to Audio to MIDI Alignment and Matching. Columbia University.

[5] Google Magenta / Keras. (2024). Music Generation with Transformer on MAESTRO. keras.io/examples.

[6] Roche, F., Hueber, T., Limier, S., & Girin, L. (2018). Autoencoders for music sound modeling: a comparison of linear, shallow, deep, recurrent and variational models.